Collaborative Traceability Management: Challenges and Opportunities

Rebekka Wohlrab, Jan-Philipp Steghöfer, Eric Knauss, Salome Maro, and Anthony Anjorin
Chalmers | University of Gothenburg
Gothenburg, Sweden
{wohlrab,anjorin}@chalmers.se, {jan-philipp.steghofer,eric.knauss,salome.maro}@cse.gu.se

***Abstract*—Traceability and trace link management are important for various reasons, including managing knowledge about a complex software system, monitoring the progress of its development, and proving that it is developed in accordance to regulations. However, it is difficult to maintain and use trace links in real-world projects where artifacts undergo constant change and multiple stakeholders are involved. In this paper, we extend the current body of knowledge on traceability management by regarding its collaborative aspects in an industrial setting. Based on 15 industrial cases and semi-structured interviews with 24 practitioners, we identify challenges involved in collaborative traceability management, and how traceability management can be used to enable collaboration. Our findings show that main challenges are boundaries between organizations and tools, a lack of common goals and responsibilities, and the difficulty of collaboratively maintaining trace links. We also identify traceability as an important facilitator for communication and knowledge management across these boundaries.**

***Index Terms*—traceability management, collaboration.**

I. Introduction

As today's software projects tend to be highly complex, traceability is necessary to ensure that connections between software artifacts can be appropriately monitored [11], [34]. Among other advantages, traceability helps stakeholders understand relationships between software artifacts, supports software and systems engineering activities such as change impact analysis [10], and improves development speed [18].

Our understanding of traceability is based on Gotel et al. [10]: Software *traceability* allows creating and using links between software artifacts, which for example allows to connect the origin of a requirement with its specification or development artifacts to each other throughout the software lifecycle. These connections are called *trace links* and link a *source artifact* to a *target artifact*. *Artifacts* can be of different *types*, such as a requirement, a model element, a line of code, or a test case. *Traceability management* is the planning, organization, and coordination of all activities related to traceability, including the creation, maintenance, and use of trace links.

While the general topic of traceability management has been addressed by a number of authors (see, e.g., [4], [30]), the work on the *collaborative* aspects of traceability management is not quite as substantial. In practice, projects are often developed by distributed teams, both software artifacts and trace links undergo constant change, and multiple stakeholders with different backgrounds are involved. The traceability research community has identified the open challenge of achieving full traceability across organizational borders, especially in large, multi-person projects [10]. To address such challenges, collaboration must be considered [15] and a collaborative tool with features for traceability management, communication, and change management is needed [33]. Such a collaborative tool should include ways to version trace links, report faulty trace links, and improve their quality in collaboration with stakeholders. However, more empirical work is required to understand challenges of traceability across boundaries in distributed scenarios and its potential [19], [27].

This paper addresses this need by reporting on an exploratory multiple case study in which we discovered collaborative challenges and the effects of traceability management on collaboration in interviews with 24 practitioners from 15 industrial cases. We investigated the following research questions:

- **RQ1:** What are practitioners' challenges with respect to collaborative traceability management?
- **RQ2:** How is collaboration affected by traceability management practices?

Based on these questions, we extend the current body of knowledge on traceability management by providing evidence empirically collected from a diverse set of sources that is independent of any concrete tool solutions. The contributions of this paper are as follows:

Challenges involved in collaborative traceability management: The main challenges are organizational and tool boundaries, a lack of common goals and responsibilities, and the difficulty of collaboratively maintaining trace links.

Potential of traceability as an enabler for collaboration: We found that traceability can support communication in distributed environments and facilitate the transparent documentation of decisions and knowledge management across boundaries. Trace links have the potential to facilitate receiving information about changes of relevant artifacts, thus supporting inter-disciplinary engineering and communication.

Based on these findings, we define *collaborative traceability management* as traceability management in multi-person projects across organizational or tool boundaries. This entails the need to share goals and responsibilities between stakeholders as well as to propagate relevant changes. It promises broad contribution to traceability management and trace link quality, if a good cost-benefit trade-off can be achieved for potential collaborators.

As a consequence of this collaborative perspective on traceability, we suggest future research that focuses on the enabling properties of traceability, in particular addressing management of information overflow and features that allow to share and discuss information about trace links.

II. Related Work: Collaborative Traceability Management in Research

Our related work is presented in two parts: first, empirical research reporting on traceability practices and challenges in industry; second, research on tools that use trace links to facilitate collaboration.

Traceability practices and challenges in industry have been investigated in several empirical studies. For instance, Bouillon et al. [3] studied traceability usage scenarios and concluded that practitioners make considerable use of traceability for requirements management (tracking the origin and rationale), project management (tracking progress), and compliance demonstration. Collaboration-related usage scenarios such as *identifying stakeholders* and *notification of stakeholders about changes* are reported, as well as an increase in the use of traceability in proportion to project size. Mäder et al. [19] report that the motivation for traceability affects how traceability management is carried out. They conclude that traceability across company boundaries is a substantial challenge that requires more attention and empirical studies to better understand traceability practices in industry. Jaber et al. [14] conducted a survey to identify a set of trace link types that are most needed for collaboration. These link types are evaluated through experiments to prove that they improve both speed and accuracy of maintenance tasks.

A study by Rempel et al. [27] observes that there is a mismatch between traceability goals and actual trace links created in practice, arguing that it is crucial to define project-specific traceability strategies based on project-specific goals. Panis [23] reports on successful implementation of traceability in industry arguing that a major challenge is a degradation of the quality of trace links over time. The focus of these studies is on traceability in general, while our study specifically investigates collaborative traceability management. The reported research also proposes further empirical studies on traceability strategies, challenges and usage in industry [19], [27].

The second relevant set of studies focuses on developing collaboration tools and frameworks based on information from trace links. Prototype tools are discussed that use traceability information to identify and notify affected users when artifacts change [9], [13]. This line of research supports RQ2, but only on the level of prototypical tools and frameworks for collaboration. In order to adopt traceability in practice, however, a tool must integrate in the tool chain and workflow present at the target companies. This is rarely achievable in pure research projects due to resource constraints and access to the required information. Cleland-Huang et al. [5] propose the need for future studies on understanding how the adoption of social collaboration tools can impact collaborative traceability management. This shows that there is a demand for empirical validation and industrial applicability of this aspect which our study aims to address. To the best of our knowledge, traceability in collaborative system development has only been directly discussed by Strašunskas [35]. He focuses on the storage, version management, and access control of artifacts in collaborative development. He proposes storing traceability "relations" in a central repository to give multiple stakeholders the opportunity to collaborate on them. The proposal, however, focuses mostly on allowing collaborative editing of software artifacts, and traceability management is not tackled directly.

III. Research Methodology

As we did not know all relevant variables of the phenomenon prior to the study, we conducted an *exploratory multiple case study* and report our findings with respect to our *exploratory research questions* [8]. Conducting a *multiple* case study allowed us to mitigate common threats to validity (discussed in Section VI-A) by applying *data (source) triangulation*, i.e., by using various sources, occasions, and individuals to explore the phenomenon from different perspectives.

a) Selection of Participants: We selected companies starting with a convenience sample [37] of project partners, which we then extended with a snow ball strategy. In this way, we made sure to include companies of different sizes, countries, and domains. Within each company, we interviewed individuals with different roles who had worked in their current roles for at least one year. Table I provides an overview of the selected cases, their countries (Sweden (SE) or Germany (DE)), their domains, characteristics of the project, distribution of sites, and the roles and experience of the interviewees.

b) Data Collection: We conducted *semi-structured interviews* [22] to collect qualitative data. An interview guide[1] was prepared including both open-ended and specific questions to not only guide the interviews' flow, but also allow the course of the conversation to develop freely. The interview guide was carefully reviewed as well as tested and refined in two pilot interviews. Mirroring techniques [22] were used to paraphrase the interviewee's statements and using them to construct subsequent questions or comments.

In total, we conducted interviews with 24 individuals from 15 cases over a period of 11 weeks. The interviews took between 45 minutes and 120 minutes, with an average length of approximately 75 minutes. We recorded and transcribed all interviews, resulting in about 475 pages of transcribed text. After the interviews, we asked our participants to give feedback on our findings via a technique referred to as *member checking*. Member checking makes the participants feel more involved in the process, understand the results, and reflect on how they can adapt their current practices [31]. Our participants confirmed the research findings.

c) Data Analysis: We followed the data analysis approach by Creswell [6], and tailored it to our needs[2]. After having thoroughly studied the collected data, we started the process

[1] https://www.dropbox.com/s/id51h5hp2g6bpez/interview_guide.pdf?dl=0

[2] For a more detailed description of our analysis method, see https://www.dropbox.com/s/rxzkaqflttrtr0y/analysis_guide.pdf?dl=0.

Table I
PARTICIPATING PROJECTS AND INTERVIEWEES IN THE CASE STUDY

Case	Country	Domain	Nr. of project members	Project duration	Distribution of sites	Interviewees
1	SE	Embedded	10–30	3–4 yrs.	One local site	Development Manager (3 yrs.) Project Manager (13 yrs.)
2	SE	Electrical equipment	$\sim$ 20	>8 yrs. of development	Multiple int. sites	Developer (10 yrs.) Team Leader Development (>10 yrs.) Project Manager (>2 yrs.)
3	DE	Automotive	>20	>2 yrs.	Multiple int. sites	Team Leader Subgroup (3 yrs.)
4	DE	Automotive	$\sim$ 30	4 yrs.	Multiple nat. sites	Quality Analyst (1 yr.)
5	SE	Telecommunications	6-8	1.5 yrs.	One local site	Project Manager (>2 yrs.) System Manager (12 yrs.)
6	SE	Automotive	15	2 yrs.	Multiple nat. sites	Developer (3 yrs.)
7	SE	Automotive	>100	3 yrs.	Multiple nat. sites	Software Architect (3 yrs.)
8	SE	Software Development	$\sim$ 4	<6 months	One local site	Application Engineer (1.5 yrs.) Chief Technical Officer (5 yrs.)
9	SE	Industrial Automation	25	<1 yr.	One local site	Software Quality Manager (>1 yr.) Head of R&D (3 yrs.)
10	DE	Automotive	10-20	1-2 yrs.	Multiple int. sites	Product Manager (14 yrs.)
11	DE	IT Services	50	5 yrs. of development	Multiple int. sites	Head of Development (9 yrs.) Developer (8 yrs.)
12	DE	Banking Self-Service Automation	10-50	6 months	Multiple int. sites	Head of Software Quality Assurance (6 yrs.) Head of Project Mgmt. Office (2.5 yrs.)
13	SE	Telecommunications	1-4	<1 yr.	Multiple int. sites	Team Leader (2 yrs.)
14	DE	Automotive	$\geq$100	2-3 yrs.	Multiple int. sites	System Software Architect (8 yrs.)
15	DE	Software Development	20-40	6 months	Multiple int. sites	Quality Manager (>1 yr.) Head of Test Management (5 yrs.)

of *coding*. Using our main research questions as a basis, we identified topics that are of interest to our research and created a priori codes for them. In the actual coding process, we read through the transcripts line by line, connecting statements from the interviews to appropriate codes using an *editing approach* [28]. We started the process based on the a priori codes defined in the previous step. New codes were created during the coding process where they evolved, were revised, merged, and split. The coding was done by one of the researchers to ensure a consistent use of the codes. We conducted a coding workshop in which we discussed the codes, their meaning, and connections in a group. We estimate that the final set of codes consists of 30-40% a priori codes. Many emergent codes are, however, sub-codes of initial codes, e.g., ten sub-codes of "Collaboration" were created and revised to adequately cover the different collaborative aspects mentioned in the interviews.

d) Reporting on Research Findings: During the analysis, we made sure that the *chain of evidence* was clear [28]. This means that all our conclusions and results needed to be comprehensible for the reader and that our procedure was documented. In a group, we discussed connections between codes and identified interesting themes to examine in more detail. *Cross-case synthesis* was applied to compare different cases, identifying similarities and differences between interview results [29, p. 69].

We iteratively structured our findings into themes, created visuals where adequate, and wrote them down. We report and discuss our findings with respect to RQ1 (challenges of collaborative traceability management) in Section IV. By analyzing challenges and discussing potential solutions with interviewees, we found that using the right strategies, traceability management might even *enable or support collaboration*. We elaborate on this theme and RQ2 in Section V. An overview of our findings and their relations is given in Figure 1.

IV. CHALLENGE: MANAGING TRACEABILITY COLLABORATIVELY

In this section, we report challenges of practitioners with respect to collaborative traceability management (RQ1).

Affected disciplines include the (sub-)disciplines of software engineering (product management, development, testing), and potentially other disciplines (e.g., systems engineering, mechanical engineering, and electrical engineering).

A. Collaboration Across Boundaries

The necessary coordination of stakeholders across disciplinary borders and tool boundaries poses a major challenge to collaborative traceability management.

1) Collaboration with other Departments: The organizational structure of most case companies is characterized by separate departments for different disciplines. There is often no centrally defined concept for traceability management that goes across disciplinary borders. In our study, only Case 14 has end-to-end traceability, from pre-requirements traceability to all phases of the development and test process. Generally, we found that it is difficult to establish common practices top-down in an entire organization. Traceability management approaches often arise in a bottom-up manner as well:

"If we are talking about full traceability, from a requirement to the details in the code and the test case result, then this affects several phases of the development process and several organizational units. [...] It requires a lot of power and energy, and good management, to set something like this up in a top-down fashion. And on the other hand, there are many smaller teams where approaches come up in a bottom-up way. [...] Of course, it can happen that something comes up here and something else there, and then these two are competing solutions..."

(a product manager from Case 10)

This difficulty of establishing common traceability practices across department borders is common in many cases. For example, a software quality manager from Case 9 stated that customer needs are not well linked to the actual implementation: "we lose track somewhere between the development and the marketing department". In Case 3, an issue is that their traceability practices are not consistent across departments:

"You can create these links already now, but [our current approach] is not consistent throughout the whole platform. Because I might draw links in one department but in another department someone might not know what belongs to what..." (a team leader from Case 3)

This is connected to the different backgrounds at different departments. Several tools, languages, and terminologies are used which makes collaboration difficult:

"That's why we have problems now, because the communication isn't so easy. When you have time differences and language and all these barriers to communicate."

(a team leader from Case 2)

We found this aspect to be especially challenging in the automotive domain. Engineers with a background in mechanics, electronics, and software engineering develop the same end-product together. A developer from Case 6 pointed out that because of the different backgrounds, it is "harder to communicate efficiently and make all the developers work together".

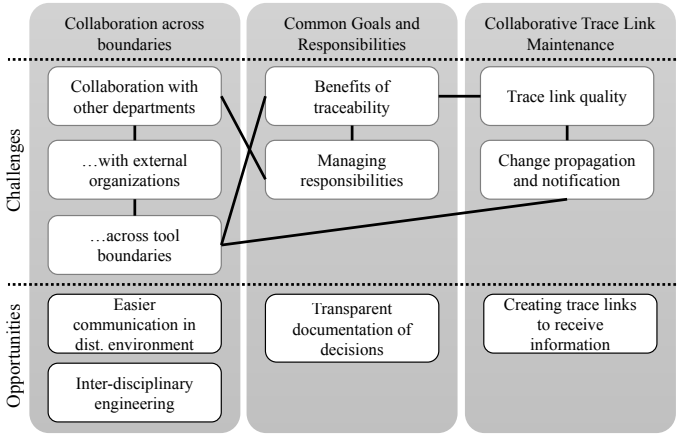

Figure 1. Challenges and opportunities in collaborative traceability management

2) Collaboration with External Organizations: Coordination across company borders is another difficulty. Few of our cases provide a way to trace to artifacts from another company.

In Case 2 and Case 8, trace links do exist that point to bugs or issues reported by the customer. Case 14 imports customers' requirements into their requirements management tool, where trace links are then created between the requirements and more detailed requirements specifications. If the requirements are updated, however, the specification must be exported and imported again, leading to inconsistent trace links.

Information exchange with external companies is typically accomplished via e-mail and manually entered into the respective tools. According to a project manager from Case 1, this makes it difficult to trace the customer's requirements back to its origin—such as who created it when.

While we discuss tool boundaries in the next section, we interpret this as a challenge that would even exist if both companies would be using the same tool.

According to the Head of Development from Case 11, a further difficulty is that suppliers have several customers and might not (want to) adapt traceability practices to one particular customer. Legal and organizational reasons also contribute to the challenge of collaboration with other companies.

3) Collaboration across Tool Boundaries: The variety of tools used in different departments contributes to the challenge of using traceability across organizational or company borders. Out of 24 interviewees, 14 stated that the heterogeneous nature of the used tools makes traceability management difficult in practice. Due to inadequate tool integration, it is difficult to create links between artifacts stored in different tools. In some cases, workaround solutions are employed using a manual mapping of IDs (such as in Case 12 or Case 5). In Case 11, the developers reference the requirements specification with its current version number as a source code comment.

B. Having Common Goals and Responsibilities

In order to successfully manage traceability, it is necessary to coordinate the beneficial use of traceability with the responsibilities of individuals. In the following, we argue that this is often difficult to achieve in practice.

1) Benefits of Traceability: Several interviewees stated that the creation and maintenance of trace links is not beneficial in itself. At the exact moment the trace link is created, it is obvious to the link creator why and how the source and target artifacts are connected. "It is just an overhead" if the creator of the trace link only cares about the current point in time, as stated by a development manager from Case 1. An application engineer from Case 8 stated that "one challenge is to make people understand the benefit".

Collaborative traceability management across organizational or tool boundaries even increases the overhead of traceability management practices and of clarifying responsibilities. Thus, a compelling reason for an organization to accept the overhead that traceability management involves must be evident. This cost-benefit trade-off was mentioned by many interviewees.

Currently, tool support for traceability management is very limited. Even in cases where traceability information is captured, it is still hard to actually use it:

> *"But that is the part where we need a better tool than we have now. Because you... All the information is there but it's kind of hard to get a big picture, an overview."*
>
> (a development manager from Case 1)

The benefits of traceability would be clearer if current uses were better supported by tools. According to the interviewees, the most common uses of trace links are: tracking the project's progress (mentioned by 15 cases) by, e.g., visualizing the passed test cases; troubleshooting (mentioned by 12 cases) to identify reasons for defects; coverage analysis (mentioned by 12 cases); change impact analysis (mentioned by 11 cases); and generating artifacts (mentioned by 9 cases) such as release notes or status reports.

Another point is that in many cases, the creators of the trace links belong to a different group than the users of the links. For instance, in Case 2, to enable a project manager to track the project's progress, the developers must create links between implementation tasks and source code commits. This is also a problem reported in other cases:

> *"In our case, the creators of the links, so the developers, actually complain a lot that they have to think of traceability. The people who really use it are support persons or project managers. So people from another team."*
>
> (a change leader from Case 5)

Especially the attitude of stakeholders plays a central role:

> *"Some people say 'I don't care, [...] I'm just interested in doing my work, why do you bother me?' It's more of that attitude sometimes."*
>
> (an application engineer from Case 8)

The benefit of traceability in an organization has to be clear to enable stakeholders to collaboratively work on the links.

2) Managing Responsibilities: Our cases have different ways of organizing the responsibilities for traceability management. In most cases, the whole team is responsible for creating trace links, which is sometimes supported by checklists and included in process definitions. Interviewees from four cases stated that traceability management tasks need to be part of the everyday work of the stakeholders in the company.

The effort required for these tasks largely depends on tool support. In Case 5, a spreadsheet is used to show the connections between artifacts in a traceability matrix. One person is working full time to keep these spreadsheets up-to-date. In Case 15, a quality manager frequently reviews the trace links and contacts the responsible persons if necessary.

Establishing and maintaining trace links has often been compared to the task of documentation, for example by a project manager from Case 1. Tasks that bring direct business value to the company or stakeholder usually have a higher priority and including traceability management in everyday work is not done if the value is not obvious as stated by a product manager from Case 10. Our theme *Benefits of Traceability* thus has a direct impact on managing responsibilities.

What should be noted in this context is that the current practices are in most cases introduced in one discipline or department. Responsibilities on a higher organizational level are often not assumed and no centrally defined concept or strategy exists to support collaboration across boundaries.

> *"The testing team does it, because they [...] care for which requirements they are covering. But it's not clear to me if somebody else really takes the responsibility for traceability in the company."*
>
> (a quality analyst from Case 4)

A software architect from Case 7 stated that in his case, it is especially difficult to find people responsible for links between disciplinary borders:

> *"So it's also an issue that there are not any persons responsible for the big picture how links are working. There are people responsible for the breakdown of requirements and there are people responsible for the software. In these boundaries not..."*
>
> (a software architect from Case 7)

In many cases, interviewees stated that because of insufficient tooling, some of the desired uses are not supported yet. An application engineer from Case 8 stated that "there should be strong tool support" to maintain and to leverage traceability to make stakeholders see the benefit and use of it. This is again more difficult when heterogeneous tools are in use.

C. Collaborative Trace Link Maintenance

Our interviewees emphasized that maintenance of trace links is relevant to ensure high trace link quality. Here, we present challenging aspects of collaborative trace link maintenance.

1) Trace Link Quality: In Section IV-B1, we pointed out that the cost-benefit trade-off of traceability management is especially critical in collaborative cases. To establish traceability management practices and encourage collaboration, the use and benefits must be made clear to both developers and managers. We found that trace links will (only) be used if trace link quality is high enough and if the links are used, there is a good chance that their quality will be maintained through collaboration.

> *"Because you need to have trust in the data. You need it to be on the level that you think it's worthwhile looking into traceability to get some benefit. If you don't trust it, then it's not used. And if it's not used, then you don't improve it."*
>
> (Chief Technical Officer from Case 8)

We found that there are several notions of what quality means and which aspects are relevant. When we asked about trace link quality, a system software architect from Case 14 stated that this was a "difficult question". It is difficult to precisely define and measure trace link quality. Our interviewees stated that especially incorrect or missing links between artifacts are an issue in practice. One interviewee explained the problem with incorrect trace links as follows:

> *"Otherwise if you try to follow up on something, then you're going to get utterly confused if you see that this*

doesn't belong together at all. I [...] prefer if there is no link instead of there being a wrong link. Because then you have to evaluate that as well, is the link correct or not? But if the link is not there, then I can search in some other way to find out what I want to know."

(a team leader from Case 2)

When asked about the automatic creation of trace links, most interviewees were hesitant. Especially the interviewees from the automotive domain stated that it would be a big issue to have incorrect links in a safety-critical system. Others stated that it would be interesting to achieve a more complete set of trace links using automatic algorithms—but rather "as an input for a final manual step", as stated by the Chief Technical Officer from Case 8.

Interestingly, the notion of correctness changes if trace links are versioned. A correct trace link then becomes "outdated" if the connected artifacts change rather than "automatically incorrect", as stated by a software architect from Case 7. However, change propagation and trace link maintenance still have to be considered to maintain high trace link quality.

2) Change Propagation and Notification: When one of the connected artifacts evolves, one must check if there are any implications to connected artifacts and the trace link. If a versioning solution exists, the trace link should be updated to connect the new versions of the artifacts. If the two artifacts should not be in relation anymore, the link should be deleted.

The vast majority of the interviewees stated that this is the most common scenario in which trace links get incorrect: An artifact is changed and connected artifacts are not updated. This is probably due to the inadequate notification:

"If you change the signal, you won't get any information about what requirements linked to it. That's probably where we introduce incorrect links."

(a software architect from Case 7)

In some cases, a whole chain of trace links and artifacts can be affected by the change of one artifact—for example, when a requirement is changed. If an organization has consistent traceability management practices, this does not only affect one area, but several ones—such as analysis, design, implementation, and testing. Propagating changes and maintaining traceability in the system is thus a collaborative task.

There are different ways to address trace link maintenance in a collaborative way. In Case 14, an official workflow is defined to take care of changes whenever an artifact is modified. All affected stakeholders need to be informed about the change and implications. This should ensure that the artifacts and trace links are updated. Other interviewees, however, reported the challenge of missing notifications when something is changed:

"We have exactly this problem: Someone changes something at the top and nobody notices it. Or not everybody [who should know] notices it."

(Head of Test Management from Case 15)

When we asked how this issue is handled, our interviewees stated that collaboration between different teams is essential:

"You have to talk about it. There are frequent meetings with [involved stakeholders] to discuss these changes and their impact. And each of them has to follow up on his or her tasks."

(Head of Software Quality Assurance from Case 12)

This is a challenge especially in today's globalized working environments:

"Especially when you work in distributed teams that do not meet in the break room every day, it is complicated."

(Head of Project Management Office from Case 12)

D. Summary and Discussion

Based on these findings, we can answer RQ1: What are practitioners' challenges with respect to collaborative traceability management? We identified three main challenges:

1) Collaboration across boundaries: To achieve full traceability in an organization, several stakeholders from different disciplines have to collaborate. Different backgrounds, terminologies, and tools make this a challenge in practice.

Our findings confirm that the organizational context is indeed one of the influencing factors of traceability as stated by Ramesh et al. [24]. Sinha et al. [33] report on the challenge of globally distributed requirements management, especially because of the difficulty of managing changes. We found that many of their reported problems also hold for traceability management. Our findings show that not only geographic separation plays a role in this context, but also the different backgrounds, used tools, and organizational separation of departments. We confirm that different stakeholder viewpoints and organizational problems are a challenge in practice, as stated by Kannenberg and Saiedian [15].

The difficulty of tool boundaries and heterogeneous tools has been reported by Kirova et al. [16]. In the design of their end-to-end software traceability tool, Asuncion et al. [2] addressed this challenge by centering the data exchange around a shared database, which is accessible by all tools. However, when different companies have to work together this is usually not feasible due to legal issues.

2) Having common goals and responsibilities: The use and benefit of traceability has to be understood by all stakeholders to justify the effort of traceability and make people work together. In many cases, the creators of links are different from the users. This requires good management of responsibilities, which is difficult to achieve, especially at disciplinary borders. Our findings thus confirm one of the reasons for poor collaboration reported by Gotel and Finkelstein [12]: they state that the invisibility of responsibilities makes it difficult to transfer information amongst parties. This issue was reported particularly for higher organizational levels between disciplinary borders.

Arkley and Riddle [1] claim that a further challenge is the lack of perceived benefit of traceability to the development. One of the results of their survey was that the creators of the trace links seem to obtain no benefit from traceability. Moreover, they had to enter the same data several times in the traceability tool. This supports our finding that tool support can have an impact on how traceability is perceived and used.

3) Collaborative trace link maintenance: Changes of artifacts also affect trace links and their quality. It is a challenge to consequently update trace links and propagate changes to other parts of the organization without the active collaboration of different departments. In line with our theme *Benefits of traceability* we note that such an approach could in fact lead to collaborative and de-centralized change management, extending the maintenance of trace links. Especially for distributed organizations, ways are needed to collaboratively create, maintain, and use trace links.

Sengupta et al. [32] identified a similar challenge for distributed software development in general—the propagation and management of requirement changes. They state that other development activities such as design and coding are often also affected. In such a scenario, traceability between these artifacts would be very useful. The general issue of change propagation could be facilitated using a suitable change notification feature.

The importance of trace link quality has been acknowledged in several studies. Rempel and Mäder [26], e.g., propose a model for assessing requirements traceability. They note that in many cases traces are created by humans and a way to assess their quality is required. Ongoing research also addresses improving the quality of manually created and thus potentially untrustworthy trace links (cf., e.g., [36]).

V. Traceability: An Enabler for Collaboration

While traceability has a number of challenges in collaborative environments, our research suggests it can also be a facilitator for collaboration. This section addresses RQ2: How is collaboration affected by traceability management practices?

A. Easier Communication in Distributed Environments

Especially in today's distributed environments with different stakeholders, offering the opportunity to collaborate in the direct context of the trace links can help to ease communication.

A product manager from Case 10 arrived at the following conclusions when considering collaboration features in a tool:

> *"But you should not forget that people work closely together in software projects. Today, it's globally, [...] distributed and in different time zones. And of course you have the possibility to send e-mails, but the plethora of emails that you receive nowadays... So it makes sense if you have close collaboration in distributed teams at distributed locations. Then having such ways to communicate that are relatively easy might be appealing."*
>
> (a product manager from Case 10)

According to our interviewees, a collaborative tracing tool could facilitate communication around a specific trace link, thus keeping information at a shared space and in a meaningful context, where it can be easily accessed by all collaborators.

B. Transparent Documentation of Decisions

When discussing using a common platform with external partners, we asked our interviewees whether this might facilitate collaboration. Two interviewees described situations where especially the collaboration with external partners was difficult because of a lack of traceability.

> *"[Using the same system as our partners] would make sense to get the end to end traceability here. As I said, yesterday we had an issue with that. Everyone had always said, 'everything works fine, the delivery will be on time'. And then three days before it wasn't as good as we thought, 'ah, so you meant it that way? I didn't know that at all.' [...] But with traceability you can see that from the start, have they understood it this way, do they know that we have [these connections to other artifacts]."*
>
> (Head of Project Management Office from Case 12)

To some extent, trace links can also be used to make sure that a team is aware of certain dependencies, or can look it up at a common place. It was suggested by a project manager from Case 1 to record requirements with their trace links during the requirements engineering phase, together with all involved stakeholders. The interviewee stated that making these connections explicit helps to make decisions in a team.

One idea for future tooling was to include a voting feature:

> *"There you can see who said 'okay' or 'not okay' and what they commented on, what they did not agree with."* (Head of Test Management from Case 15)

The use of the voting feature could help to support decision making and ensure that the stakeholders are aware of connections between artifacts.

C. Creating Trace Links to Receive Information

An idea that came up in the interviews was to include a change notification feature. Whenever an artifact changes, such a feature could send the person responsible for the connected artifact(s) an automatic notification to check the trace link. It was stated that such a feature would be indeed beneficial:

> *"When the requirements engineer changes something and I notice it through the chain [of trace links] then this facilitates collaboration. Because if one forgets to tell the others about a change then nobody notices it. And [change notification] would have created a big advantage."*
>
> (Head of Software Quality Assurance from Case 12)

Several interviewees see the potential of using trace links as a way to collaborate. This could also change what trace links are created: creating a trace link between a source and a target artifact makes sense if changes to the source artifact should be noticed by the person responsible for the target artifact.

D. Inter-Disciplinary Engineering

Software engineers rarely work in isolation. In many cases, they need to coordinate with systems and hardware engineers to integrate aspects from different disciplines into one end product. Trace links can help to pass the changes made in one discipline to other affected parts:

> *"Traceability can help at the point where you have changes in one domain. [...] For that traceability is of course important. To say, for example, I have a software*

port... If it is inside the software I can change it as I want. But if it is the hardware-software interface, someone else has to notice. And to get exactly this information—which ports I can change without consultation and which I can't—that's what traceability is important for."

(a system software architect from Case 14)

Traceability can help to see what system parts are affected by changes and whether other disciplines must be consulted.

E. Summary and Discussion

In this section, we summarize our results for RQ2: How is collaboration affected by traceability management practices? We found that collaborative traceability management offers four opportunities for easier collaboration:

1) Easier communication in distributed environments: Evaluating their distributed requirements management tool, Sinha et al. [33] found that contextual collaboration is a popular feature. Our findings confirm that especially in distributed teams, it is beneficial to have such features in place, also for collaborative traceability management. What should be noted here is that the users need to be comfortable with the provided tool solutions. It might be interesting to plug in existing communication mechanisms that are known to the user, as stated by Sinha et al. [33]. We claim that usability and stakeholders' familiarity with such features have a high impact on their acceptance in practice.

2) The transparent documentation of decisions: We found that trace links can help to ensure awareness of dependencies in multi-person projects. One idea that came up in our interviews was to use a voting feature to record the approval of changes at a common place. Having trace links in place can actually support tasks like decision making in a transparent way.

The issue of knowledge management has been identified as a problem of global software development before. Damian and Zowghi suggest to use a repository of project artifacts and to rigorously maintain it [7]. Our findings confirm the results of Ramesh's empirical study [25] that traceability can help to facilitate knowledge management processes.

3) Creating trace links to receive information: The change notification feature could actually help to collaborate as the responsible stakeholders receive notifications about changes directly through the tool. Helming et al. [13] created a model-based change awareness approach that uses trace links to identify which users to notify. They evaluated it in a case study and state that it is useful in practice. Our findings confirm that practitioners are indeed interested in such features.

4) Inter-disciplinary engineering: Traceability helps to capture dependencies between different parts of a system. This allows the users to identify whether changes can be made without consulting other disciplines. Königs et al. [17] analyzed current systems engineering practices and concluded that traceability can support the interaction and communication in these inter-disciplinary scenarios. They present two tools supporting traceability management in systems engineering contexts. Our results confirm that practitioners find it promising to use traceability features in systems engineering in order to facilitate collaboration.

VI. Discussion

In this section, we discuss our research findings and point out implications of this work for research and in practice.

An overview of our findings, according to research question, theme, and topic can be found in Table II. There, we also list the cases in which the issue was discussed most prominently as well as the relevant related literature as referenced in the respective discussion sections for the research questions.

We pointed out which challenges practitioners have in the context of collaborative traceability management. Until now, collaborative aspects have been neglected when traceability management practices were analyzed [10], [19]. We identified that coordination across disciplinary borders, a lack of common goals and responsibilities, and collaborative maintenance of trace links are complicating factors.

Common goals and clear benefits are needed for organizations to facilitate coordination across boundaries. Tool suppliers can contribute to overcome tool barriers, working towards common standards like OSLC[3]. We found that tool boundaries and the lack of adequate tool support cannot be blamed for all practical issues. A more essential issue is that practitioners do not always see the benefit of traceability, and thus do not make use of it in their organizations. Communicating the cost-benefit trade-off is therefore imperative for the acceptance of traceability in practice. We showed how traceability can be used as an enabler for collaboration, supporting knowledge management and facilitating communication especially in distributed environments. Concretely, we found potential benefit in four areas: Easier communication in distributed environments, transparent documentation of decisions, the motivation of creating trace links to receive information about changes, and using traceability to support inter-disciplinary engineering.

Our results allow both practitioners and researchers deeper insights into current challenges and influencing factors of collaborative traceability management. Practitioners can use this study as a starting point to reflect on their uses of traceability, the benefits they see, and ways to establish more targeted, efficient collaborative traceability management practices.

A. Research Validity

Maxwell describes six relevant threats to validity in qualitative research [20], [21]. These are discussed in the following:

a) Descriptive Validity: It is not possible to completely avoid this threat, but we used member checking [31] as one mitigation strategy. We created interview transcripts to be able to analyze our interviews and the interviewees' statements as they said it. However, we did not capture aspects such as ironic tones, pauses, or gestures. This might have led to a wrong interpretation of the gathered data. We were, however, always able to clarify confounding parts in the transcripts by referring to the interviews' recording.

[3]http://open-services.net

Table II
OVERVIEW OF RESEARCH FINDINGS

Theme	Topic	Main Relevant Cases	Related Literature
RQ1 – Challenge: Manage Traceability Collaboratively			
Collaboration across boundaries	Collaboration with other Departments	2, 3, 6, 9, 10, 12	[15], [24], [33]
	Collaboration with External Organizations	1, 2, 8, 7, 11, 12, 14	
	Collaboration across Tool Boundaries	5, 11, 12 and 11 others	[2], [16]
Common Goals and Responsibilities	Benefits of Traceability	1, 2, 5, 8	[1]
	Managing Responsibilities	1, 4, 5, 7, 8, 9, 10, 14, 15	[12]
Collaborative Maintenance of Trace Links	Trace Link Quality	2, 3, 7, 8, 12, 14	[26], [36]
	Change Propagation and Notification	6, 7, 12, 14, 15	[32]
RQ2 – Traceability: An Enabler for Collaboration			
Easier Communication in Distributed Environments		10, 15	[33]
Transparent Documentation of Decisions		1, 7, 12, 15	[7], [25]
Creating Trace Links to Receive Information		7, 12	[13]
Inter-Disciplinary Engineering		14	[17]

b) Interpretative Validity: At the beginning of the interviews, we ensured that there was a common understanding of the used terminology and the concept of traceability. We conducted the majority of the interviews in the interviewees' native languages or made sure that the interviewee had an advanced English level to prevent misunderstandings due to language issues. However, we translated selected quotes from German to English. While we aimed to keep the meaning as close as possible to the original statements, there is a potential for translation inaccuracies affecting our interpretation.

c) Theoretical Validity: During the course of this study, we built preconceptions that we expected the data to support, thus threatening validity by potentially interpreting data from this biased perspective. We might have treated data differently that did not support them. To mitigate these threats, we collected feedback from researchers that were not directly involved. This was done throughout the process, with a special focus on the analysis phase and the design of our research method.

d) Researcher Bias: It is impossible to eliminate the researchers' theories, background, and values, and their impact when arriving at qualitative conclusions. As opposed to a case study evaluating a developed method or tool, we had nothing to prove or gain from this study. As mentioned above, we received feedback from fellow researchers and discussed analysis strategies and preliminary results.

e) Reactivity: Another threat that can hardly be avoided is the influence of the researcher on the setting or participants of the study [21]. Factors such as tone, listening strategies, or spontaneous reactions influence the interviews. We tried to design our interview guide with as few misleading or biased questions as possible. Test interviews and reviews of the interview guide helped to identify and correct biased questions.

f) Generalizability: Generalizability is connected to the transferability of our findings to other situations or cases than the ones we have directly studied [20]. The goal of qualitative research is more the particular description in context of specific cases rather than generalizability [6].

Triangulation [29] allowed us to examine the studied phenomenon traceability from different angles. However, we specifically asked for interviewees with knowledge in the area of traceability. This affected our sampling, since we conducted interviews with individuals who probably already had rather positive connections to the topic.

B. Future Work

Future work should validate our findings and build upon the identified opportunities. This study is an exploratory case study in which we conducted interviews with 24 practitioners from industry. More empirical studies are needed to validate the presented findings. For instance, a quantitative study could be conducted validating the identified challenges using a survey.

We found that until now, practitioners do not use automatic approaches for traceability management, whereas there exists a substantial amount of research in this area. Our findings show that practitioners would use input from automatic algorithms as a suggestion—provided that it is of sufficiently high quality. One possible direction to make automatic trace link maintenance more useful for practitioners that is in line with our findings is to provide *decision support* to developers by making feasible suggestions for potential trace links. The challenge is not to overwhelm the user with potential links and clearly communicate the reasoning behind each suggestion.

A more pressing concern, however, is how to enable collaboration on trace links to maintain a high trace link quality. Practitioners need to be notified about changes without being overwhelmed by them. Selecting a manageable number of relevant changes is therefore of the essence. In addition, traceability tools must enable direct communication about the trace links within the tool. The voting feature mentioned above is one possibility. Giving practitioners the possibility to comment and discuss specific trace links and associating that information with the links is another area to explore.

Our findings suggest that the benefit of trace links has to be clear for practitioners to create, maintain, and use them. Future work could investigate how the benefit of traceability can be better conveyed to practitioners. Analyzing usage data of trace links in projects could help to identify which trace links are needed to bring added value to the organization. Our ideas to use traceability as an enabler for collaboration imply another potential benefit, if sufficient tool support can be provided.

VII. Conclusion

In this study, we explored challenges and requirements of practitioners in 15 industrial cases in an exploratory multiple case study based on 24 interviews with software engineering stakeholders in Sweden and Germany. Challenges of managing traceability collaboratively include distributed organizational structures, tool boundaries, and the lack of common goals and responsibilities. Especially the collaborative maintenance of trace links is difficult as artifact changes are often not sufficiently communicated to affected stakeholders.

We found the cost-benefit trade-off of traceability management to be crucial. Practitioners have to see the usefulness and benefits of traceability. As a consequence, collaboration will be improved and also trace link quality will be regarded as important. Our findings on collaborative traceability management indicate that the challenges in traceability management and distributed software engineering do not necessarily exacerbate each other. In contrast, we found that traceability can enable collaboration and facilitate knowledge management. Traceability can facilitate communication in distributed environments, allow the transparent documentation of decisions, and motivate the creation of trace links in order to receive information about changes to connected artifacts. Finally, traceability can support coordination across disciplines in systems engineering contexts.

Our results allow practitioners to critically reflect on their traceability management practices and their potential to improve collaboration. The research community benefits from this new perspective on traceability management, taking collaborative aspects into consideration. Our findings should be validated in future empirical studies, especially with respect to the acceptance and utility of the suggested tool features in practice. Other areas include exploration of communication and collaboration features focused on trace links.

Acknowledgements

This work was supported by the Wallenberg Autonomous Systems Program (WASP), Amalthea4Public, and NGEA.

References

[1] P. Arkley and S. Riddle. Overcoming the traceability benefit problem. In *RE'05*, pages 385–389, 2005.

[2] H. U. Asuncion, F. François, and R. N. Taylor. An end-to-end industrial software traceability tool. In *ESEC/FSE*, pages 115–124, 2007.

[3] E. Bouillon, P. Mäder, and I. Philippow. A survey on usage scenarios for requirements traceability in practice. In J. Doerr and A. L. Opdahl, editors, *Requirements Engineering: Foundation for Software Quality*, pages 158–173, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[4] J. Cleland-Huang, O. Gotel, and A. Zisman, editors. *Software and Systems Traceability*. Springer-Verlag London Limited, 2012.

[5] J. Cleland-Huang, O. C. Z. Gotel, J. Huffman Hayes, P. Mäder, and A. Zisman. Software traceability: Trends and future directions. In *Proc. of the Future of Software Engineering (FOSE'14)*, pages 55–69, 2014.

[6] J. W. Creswell. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. Sage Publications Ltd., 3 edition, 2008.

[7] D. Damian and D. Zowghi. RE challenges in multi-site software development organisations. *RE*, 8(3):149–160, 2003.

[8] S. Easterbrook, J. Singer, M.-A. Storey, and D. Damian. Selecting empirical methods for software engineering research. *Guide to Advanced Empirical Software Engineering*, pages 285–311, 2008.

[9] M. C. Figueiredo and C. R. De Souza. Wolf: Supporting impact analysis activities in distributed software development. In *5th Int. Ws. on Cooperative and Human Aspects of SE*, pages 40–46, 2012.

[10] O. Gotel, J. Cleland-Huang, et al. Traceability fundamentals. In J. Cleland-Huang, O. Gotel, and A. Zisman, editors, *Software and Systems Traceability*, pages 3–22. Springer London, 2012.

[11] O. Gotel, J. Cleland-Huang, J. Hayes, A. Zisman, A. Egyed, P. Grunbacher, and G. Antoniol. The quest for ubiquity: A roadmap for software and systems traceability research. In *RE'12*, pages 71–80, 2012.

[12] O. Gotel and A. C. Finkelstein. An analysis of the requirements traceability problem. In *RE'94*, pages 94–101, 1994.

[13] J. Helming, M. Koegel, et al. Traceability-based change awareness. In *MODELS*, pages 372–376, 2009.

[14] K. Jaber, B. Sharif, and C. Liu. A study on the effect of traceability links in software maintenance. *IEEE Access*, 1:726–741, 2013.

[15] A. Kannenberg and H. Saiedian. Why software requirements traceability remains a challenge. *The Journal of Defense Software Engineering*, 22(7):14–19, 2009.

[16] V. Kirova, N. Kirby, D. Kothari, and G. Childress. Effective requirements traceability: Models, tools, and practices. *Bell Labs Technical Journal*, 12(4):143–157, 2008.

[17] S. F. Königs, G. Beier, A. Figge, and R. Stark. Traceability in systems engineering — review of industrial practices, state-of-the-art technologies and new research solutions. *Advanced Engineering Informatics*, 26(4):924–940, 2012.

[18] P. Mäder and A. Egyed. Do developers benefit from requirements traceability when evolving and maintaining a software system? *Empirical Software Engineering*, 20(2):413–441, 2015.

[19] P. Mäder, O. Gotel, and I. Philippow. Motivation matters in the traceability trenches. In *RE'09*, pages 143–148, 2009.

[20] J. Maxwell. Understanding and validity in qualitative research. *Harvard Educational Review*, 62(3):279–301, 1992.

[21] J. Maxwell. *Qualitative Research Design: An Interactive Approach*. Applied Social Research Methods. SAGE Publications, 2012.

[22] M. D. Myers and M. Newman. The qualitative interview in IS research: Examining the craft. *Information and Organization*, 17(1):2–26, 2007.

[23] M. C. Panis. Successful deployment of requirements traceability in a commercial engineering organization...really. In *RE'10*, pages 303–307, Sept 2010.

[24] B. Ramesh. Factors influencing requirements traceability practice. *Communications of the ACM*, 41(12):37–44, 1998.

[25] B. Ramesh. Process knowledge management with traceability. *IEEE Software*, 19(3):50–52, 2002.

[26] P. Rempel and P. Mäder. A quality model for the systematic assessment of requirements traceability. In *RE'15*, pages 176–185, Aug 2015.

[27] P. Rempel, P. Mäder, and T. Kuschke. An empirical study on project-specific traceability strategies. In *RE'13*, pages 195–204, 2013.

[28] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *ESE*, pages 131–164, 2009.

[29] P. Runeson, M. Höst, A. Rainer, and B. Regnell. *Case Study Research in Software Engineering*. John Wiley & Sons, Inc., Hoboken, NJ, 2012.

[30] I. Santiago, Á. Jiménez, J. M. Vara, V. De Castro, V. A. Bollati, and E. Marcos. Model-driven engineering as a new landscape for traceability management: A systematic literature review. *IST*, 54:1340–1356, 2012.

[31] C. B. Seaman. Qualitative methods in empirical studies of software engineering. *IEEE TSE*, 25(4):557–572, 1999.

[32] B. Sengupta, S. Chandra, et al. A research agenda for distributed software development. In *ICSE*, pages 731–740, 2006.

[33] V. Sinha, B. Sengupta, et al. Enabling collaboration in distributed requirements management. *IEEE Software*, 23(5):52–61, 2006.

[34] G. Spanoudakis and A. Zisman. Software traceability: A roadmap. In *Handbook of Software Engineering and Knowledge Engineering*, volume 3, pages 395–428. World Scientific Publishing, 2005.

[35] D. Strašunskas. Traceability in collaborative systems development from lifecycle perspective. In *1st Int. Ws. on Traceability*, pages 54–60, 2002.

[36] S. K. Sundaram, J. H. Hayes, A. Dekhtyar, and E. A. Holbrook. Assessing traceability of software engineering artifacts. *RE*, 15(3):313–335, 2010.

[37] C. Wohlin, M. Höst, and K. Henningsson. *Empirical Methods and Studies in Software Engineering: Experiences from ESERNET*, chapter Empirical Research Methods in Software Engineering, pages 7–23. Springer Berlin Heidelberg, Berlin, Heidelberg, 2003.